

W07 — Golden-Replay + Calibration Harness

Status: implementation-ready **Owner:** LLM Judge workstream **Unblocks:** PRD S10 Vivipedia back-test ($\kappa \geq 0.7$ gate)
Depends on: - W3 judge_calibration contract (kappa + delta shape; this plan mirrors that contract and re-uses it where possible) - Existing golden-export versioning (supabase/migrations/20260421_0013_golden_exports.sql) - POST /v1/judge-config/test (one-off path; we generalize its invocation)

1. Problem & Goal

QA needs to answer "will this new judge config agree with our humans?" before rolling it out. Today: - GoldenDataset.tsx is read-only (no replay button). - JudgeConfig.tsx has a *single-claim* "Test with Sample" modal — not a population test. - POST /v1/judge-config/test runs one claim at a time with no aggregation, no κ , no persistence.

Goal: re-run an entire golden export through a candidate judge config, persist per-claim + aggregate results, surface κ per metric with traffic-light, and expose disagreement triage for rubric/prompt iteration. Runs must be reproducible (pinned judge_config_version + golden_export_id).

Non-goals: live-traffic shadow testing, A/B routing, auto-promotion. Human stays in the loop.

2. Data model

2.1 Migration supabase/migrations/20260422_0001_calibration_runs.sql

```
-- Harness for golden-replay against a specific judge config.
create table calibration_runs (
  id                uuid primary key default gen_random_uuid(),
  project_id        uuid not null references projects(id) on delete cascade,
  judge_config_id    uuid not null references judge_configs(id),
  config_version     int  not null,                -- pinned snapshot
  golden_export_id   uuid not null references golden_exports(id),
  status            text not null default 'queued'
                  check (status in ('queued', 'running', 'succeeded', 'failed', 'cancelled')),
  started_at        timestampz,
  finished_at       timestampz,
  total             int  not null default 0,
  processed         int  not null default 0,
  passed            int  not null default 0,        -- delta <= tolerance, all metrics
  failed            int  not null default 0,        -- judge error OR delta > tolerance
  -- per-metric aggregates (fc=factual_correctness, ca=claim_alignment,
  -- cs=citation_support, cp=completeness, rl=relevance) – mirror W3 kappa shape
  mean_delta_fc     numeric(6,4),
  mean_delta_ca     numeric(6,4),
  mean_delta_cs     numeric(6,4),
  mean_delta_cp     numeric(6,4),
  mean_delta_rl     numeric(6,4),
  kappa_fc          numeric(6,4),
```

```

kappa_ca          numeric(6,4),
kappa_cs          numeric(6,4),
kappa_cp          numeric(6,4),
kappa_rl          numeric(6,4),
error_message     text,
created_by        uuid not null references users(id),
created_at        timestampz not null default now()
);

create index idx_calibration_runs_project on calibration_runs(project_id, created_at desc);
create index idx_calibration_runs_status on calibration_runs(status)
  where status in ('queued','running');
create index idx_calibration_runs_config on calibration_runs(judge_config_id, config_version);

-- One row per claim replayed in a run.
create table calibration_results (
  id                uuid primary key default gen_random_uuid(),
  calibration_run_id uuid not null references calibration_runs(id) on delete cascade,
  claim_id          uuid not null references claims(id),
  human_fc          int, judge_fc int, delta_fc int,
  human_ca          int, judge_ca int, delta_ca int,
  human_cs          int, judge_cs int, delta_cs int,
  human_cp          int, judge_cp int, delta_cp int,
  human_rl          int, judge_rl int, delta_rl int,
  max_abs_delta     int generated always as (greatest(
    abs(coalesce(delta_fc,0)), abs(coalesce(delta_ca,0)),
    abs(coalesce(delta_cs,0)), abs(coalesce(delta_cp,0)),
    abs(coalesce(delta_rl,0)))) stored,
  judge_latency_ms  int,
  judge_tokens_in   int,
  judge_tokens_out  int,
  judge_error       text,
  judge_raw         jsonb,          -- full judge payload for audit
  created_at        timestampz not null default now(),
  unique (calibration_run_id, claim_id)
);

create index idx_calibration_results_run_delta
  on calibration_results(calibration_run_id, max_abs_delta desc);
create index idx_calibration_results_claim
  on calibration_results(claim_id);

-- RLS: project-scoped; writes restricted to PM+ADMIN.
alter table calibration_runs  enable row level security;
alter table calibration_results enable row level security;
-- policies mirror golden_exports pattern (see migration _0013).

```

Rationale: - Five metrics hard-coded as columns (not EAV) so κ and delta queries stay trivial SQL. Matches the judge output shape the backend already returns. - config_version pinned at enqueue time → re-running same run id is reproducible even if the parent judge_configs row mutates. - max_abs_delta stored-generated → cheap ORDER BY max_abs_delta DESC for worst-disagreement triage. - Unique (run, claim) prevents double-writes from worker retries.

2.2 Relationship to W3 judge_calibration

W3 stores *historical* calibration snapshots attached to a judge_config's lifecycle (for trend charts). W07 stores *explicit replay runs* triggered by QA. On run success the worker SHOULD insert a summary row into judge_calibration (FK to calibration_runs.id) so the W3 dashboard picks it up for free. Exact column list deferred to W3 plan; contract used here: {judge_config_id, config_version, kappa_{fc,ca,cs,cp,rl}, sample_size, source_run_id}.

3. Backend

3.1 Worker backend/src/workers/calibration-worker.ts

Pattern: reuse the existing pg_notify/listen worker shape (same as export worker). One long-lived process; LISTEN calibration_run_enqueued; on notify, SELECT ... FOR UPDATE SKIP LOCKED the oldest queued run.

Pseudocode:

```
async function processRun(runId: string) {
  const run = await db.one(`
    update calibration_runs
      set status='running', started_at=now()
    where id=$1 and status='queued'
    returning *`, [runId]);

  // Snapshot config at run time.
  const cfg = await loadJudgeConfigAt(run.judge_config_id, run.config_version);

  // Iterate the golden export's claim manifest.
  const claims = await loadGoldenExportClaims(run.golden_export_id);
  await db.none(`update calibration_runs set total=$1 where id=$2`,
    [claims.length, runId]);

  const concurrency = Number(env.CALIBRATION_CONCURRENCY ?? 4);
  await pMap(claims, async (c) => {
    try {
      const judge = await invokeJudge(cfg, c); // reuses judge service
      const deltas = computeDeltas(c.humanScores, judge.scores);
      await db.none(`insert into calibration_results (...) values (...)`, [...]);
    } catch (err) {
      await db.none(`insert into calibration_results (... , judge_error) values (...)`, [...]);
    } finally {
      await db.none(`
        update calibration_runs
          set processed = processed + 1,
              passed    = passed + case when $2 then 1 else 0 end,
              failed    = failed + case when $2 then 0 else 1 end
        where id=$1`, [runId, ok]);
      await db.none(`notify calibration_run_progress, $1`, [JSON.stringify({runId, processed})]);
    }
  }, { concurrency });

  // Aggregate.
```

```

const agg = await db.one(`
  select
    avg(abs(delta_fc))::numeric(6,4) as mean_delta_fc,
    ...,
    /* kappa computed in app code, not SQL */
    array_agg(row(human_fc, judge_fc)) as pairs_fc,
    ...
  from calibration_results where calibration_run_id=$1`, [runId]);

const kappas = {
  fc: cohensKappa(agg.pairs_fc),
  ca: cohensKappa(agg.pairs_ca),
  ...
};

await db.none(`
  update calibration_runs
    set status='succeeded', finished_at=now(),
        mean_delta_fc=$2, ..., kappa_fc=$7, ...
  where id=$1`, [runId, ...]);

await writeW3Snapshot(run, kappas, claims.length); // best-effort
await db.none(`notify calibration_run_progress, $1`,
  [JSON.stringify({runId, status:'succeeded'})]);
}

```

Kappa implementation — backend/src/lib/kappa.ts:

```

/** Cohen's weighted  $\kappa$  (linear weights) for ordinal ratings 1..5. */
export function cohensKappa(pairs: Array<[number, number]>, maxLabel = 5): number {
  const n = pairs.length;
  if (n === 0) return NaN;
  const size = maxLabel;
  const obs = Array.from({length:size}, ()=>new Array(size).fill(0));
  const rowT = new Array(size).fill(0);
  const colT = new Array(size).fill(0);
  for (const [a,b] of pairs) {
    if (a==null || b==null) continue;
    obs[a-1][b-1]++; rowT[a-1]++; colT[b-1]++;
  }
  let po = 0, pe = 0, wMax = size - 1;
  for (let i=0;i<size;i++) for (let j=0;j<size;j++) {
    const w = 1 - Math.abs(i-j)/wMax; // linear weight
    po += w * (obs[i][j]/n);
    pe += w * ((rowT[i]/n) * (colT[j]/n));
  }
  return (po - pe) / (1 - pe);
}

```

Test matrix (vitest, backend/test/lib/kappa.test.ts): - perfect agreement $\rightarrow \kappa == 1$ - total disagreement (swapped labels) $\rightarrow \kappa < 0$ - random (chance) $\rightarrow \kappa \approx 0$ within tolerance - single rater pair $\rightarrow$ NaN-safe - nulls skipped - linear weighting: off-by-one disagreement weighted less than off-by-four

3.2 API route backend/src/routes/calibration.ts

Register under /v1:

Method	Path	Role	Purpose
POST	/v1/calibration-runs	PM, ADMIN	enqueue; body {projectId, judgeConfigId, goldenExportId}; 202 → {id, status:'queued'}
GET	/v1/calibration-runs	PM, QA, ADMIN	list by project; filters status, judgeConfigId, paginated
GET	/v1/calibration-runs/:id	PM, QA, ADMIN	full row incl. aggregates
GET	/v1/calibration-runs/:id/results	PM, QA, ADMIN	query metric=fc\ ca\ cs\ cp\ rl\ all, sort=delta_desc\ delta_asc\ claim, limit, cursor; joins claims for title/snippet
GET	/v1/calibration-runs/:id/export.csv	PM, QA, ADMIN	streamed CSV (no row cap for ≤ 5k claims; chunked otherwise)
POST	/v1/calibration-runs/:id/cancel	PM, ADMIN	only when status in ('queued', 'running')
GET	/v1/calibration-runs/:id/stream	PM, QA, ADMIN	SSE; proxies pg_notify calibration_run_progress events for this id

Validation: - Enqueue verifies judgeConfigId and goldenExportId both belong to projectId. - Snapshots config_version = (select version from judge_configs where id=\$1) at insert time. - Rate-limit: at most one queued|running run per (judgeConfigId, goldenExportId) tuple (409 otherwise) — prevents accidental double-clicks.

Audit: every POST writes an audit_log entry (action='calibration_run.enqueue' | 'calibration_run.cancel').

3.3 Reuses / refactors (no source edits in this plan; note for implementer)

- POST /v1/judge-config/test internals factor out into services/judge-invoker.ts so the worker calls the same function path.
- golden_exports → need a helper loadGoldenExportClaims(exportId) that returns [{claim_id, humanScores:{fc,ca,cs,cp,rl}, article, citations}]. If not already present in export payload, read from claim_scores joined on the snapshot manifest.

4. Frontend

4.1 Component tree

```
src/components/CalibrationHarness/  
  CalibrationHarness.tsx      # page shell, route target  
  CalibrationHarness.module.css  
  NewRunForm.tsx              # judgeConfig version picker + goldenExport picker
```

RunList.tsx	# recent runs (status + κ chips)
RunDetail.tsx	# progress + aggregates + tabs
KappaTrafficLight.tsx	# <KappaChip value={0.72} />
DeltaHistogram.tsx	# recharts bar: x=-4..+4, y=count, per metric
DisagreementTable.tsx	# top-20 worst, sortable, drill-down link
useCalibrationStream.ts	# SSE hook → live progress
api.ts	# fetch wrappers typed from backend DTOs
index.ts	

4.2 Flow

1. NewRunForm

2. Dropdown: active + historical judge configs (label: "v{version} – {label} ({createdAt})").
3. Dropdown: golden exports scoped to project (label: "Export v{n} – {claimCount} claims").
4. Shows estimated cost: $\text{claimCount} \times \text{mean_tokens} \times \text{provider_rate}$ (pull provider rate from `llm_providers`).

5. "Run calibration" → POST `/v1/calibration-runs` → routes to `/calibration/:id`.

6. RunDetail

7. Header: status badge, config name+version, export version, "Cancel" (PM/ADMIN, while running).
8. Progress bar bound to SSE processed/total.
9. Aggregates panel: 5 <KappaChip> (fc, ca, cs, cp, rl) + mean $|\Delta|$ per metric.

10. Tabs:

- **Overview** — stacked <DeltaHistogram> per metric.
- **Disagreements** — <DisagreementTable> ordered by `max_abs_delta desc`; row → <ClaimReview `claimId=...`> drawer (reuses existing component in read-only mode so reviewers can see why judge diverged).
- **Raw** — JSON viewer for selected row's `judge_raw` (admin-only).

11. "Export CSV" → GET `.../export.csv`.

4.3 KappaTrafficLight

```
const bucket = (k:number) =>
  isNaN(k) ? 'muted' : k >= 0.7 ? 'green' : k >= 0.5 ? 'amber' : 'red';
```

Shows numeric value + coloured pill. Tooltip explains the thresholds and links to PRD S10 acceptance note.

4.4 SSE hook

```
export function useCalibrationStream(runId: string) {
  const [state, set] = useState<RunState|null>(null);
  useEffect(() => {
    const es = new EventSource(`/v1/calibration-runs/${runId}/stream`, {withCredentials:true});
    es.onmessage = (e) => set(JSON.parse(e.data));
    return () => es.close();
  }, [runId]);
  return state;
}
```

Fallback: on stream drop, poll GET `/v1/calibration-runs/:id` every 3s while status is `queued|running`.

4.5 Nav + route gate

- Add Calibration entry to the admin/PM sidebar in `src/components/Nav/*` (next to Golden Dataset).
- Route: `/calibration` (list), `/calibration/:id` (detail).
- Gate via existing role HOC: roles PM and ADMIN only. QA gets read-only detail access (no "Run calibration" button, no cancel).

5. Testing

5.1 Vitest — backend

- `backend/test/lib/kappa.test.ts` — matrix from §3.1.
- `backend/test/workers/calibration-worker.test.ts` — fake judge-invoker; asserts:
 - deltas persisted correctly;
 - kappa aggregates match manual computation on 10-claim fixture;
 - judge error on one claim → failed incremented, run still completes;
 - cancel mid-run → `status=cancelled`, partial results preserved.
- `backend/test/routes/calibration.test.ts` — 403 for ANNOTATOR, 409 on duplicate enqueue, body validation.

5.2 Playwright — E2E (`tests/e2e/calibration.spec.ts`)

- PM logs in → creates run against seeded golden export → progress bar → completes → verifies κ chips render with expected colour → exports CSV → asserts file header row.
- QA logs in → detail page readable → no Run button.
- ANNOTATOR → route redirects to `/`.

Seed extension: `tests/fixtures/seed-all-features.js` already provisions a golden export; add a fixture judge config whose mock invoker returns deterministic scores so the E2E κ is exactly 1.0.

6. Rollout

1. Land migration behind gate (runs on next deploy via `npm run migrate`).
2. Ship worker + API first (flag `CALIBRATION_HARNESS_API=on`).
3. Ship frontend behind `VITE_FEATURE_CALIBRATION=on`.
4. Dog-food on the v5.09 seed project; confirm $\kappa \geq 0.7$ on hand-built agreement fixture.
5. Flip flags default-on for S10.

Ops: - Worker is a separate Node process (same Dockerfile, new command `node dist/workers/calibration-worker.js`). Railway: add a second service or bundle under existing worker dispatcher. - Env: `CALIBRATION_CONCURRENCY` (default 4), `CALIBRATION_TIMEOUT_MS` (default 60_000), `CALIBRATION_MAX_CLAIMS_PER_RUN` (default 5000 — hard cap to prevent runaway cost).

7. Observability

- Log per claim: `{runId, claimId, latencyMs, tokensIn, tokensOut, delta_max}`.

- Metrics (stdout-JSON for Railway log-based metrics):
- `calibration.run.duration_ms` histogram
- `calibration.claim.duration_ms` histogram
- `calibration.run.kappa` gauge (tagged by metric)
- Alert: `status='failed'` → `audit_log` + Slack webhook (reuse existing `notify_admin`).

8. Risks & mitigations

#	Risk	Mitigation
1	Judge LLM cost spike on a 5k-claim replay	<code>CALIBRATION_MAX_CLAIMS_PER_RUN</code> ; cost preview in <code>NewRunForm</code> ; cancel button
2	Non-reproducibility (<code>judge_config</code> mutated after enqueue)	Snapshot <code>config_version</code> + store resolved prompt/model in <code>judge_raw</code> on first claim
3	κ is misleading with tiny samples	Show <code>sample_size</code> ; hide traffic-light when $n < 30$, show warning pill instead
4	<code>pg_notify</code> payload >8KB truncation	Only send <code>{runId, processed, total, status}</code> — never full rows
5	Worker crash mid-run leaves <code>status='running'</code> forever	Heartbeat column <code>last_heartbeat_at</code> ; sweeper job marks stale runs failed after 10× timeout
6	Unique (<code>run, claim</code>) blocks legitimate retries after transient failure	Retry path <code>UPDATEs</code> on conflict, resetting <code>judge_error=null</code>

9. Acceptance checklist (per-PR)

- ☐ Migration applies cleanly on fresh embedded-postgres and on production pooler URL.
- ☐ `npm run test` (backend) passes incl. new kappa + worker tests.
- ☐ `npm run test:e2e` passes the new calibration spec.
- ☐ `npm run lint` clean.
- ☐ Manual smoke: enqueue a run, watch SSE, see κ chips, CSV export opens in Excel.
- ☐ PRD traceability: link PR to US-A0?? (add story row in next PRD bump — defer to PO).
- ☐ Audit log shows `calibration_run.enqueue` for the smoke run.

10. Out of scope (future work)

- Multi-config bake-off (run N configs against 1 export, compare side-by-side).
- Auto-promote judge config when $\kappa \geq 0.7$ for M consecutive runs.
- Per-annotator κ (inter-rater instead of judge-vs-human).
- Slice analysis (κ by topic / domain / article length).

End of W07 plan.